

```

1 // Comments marked by initials ER, KP, KL, DS, EC
2 //#####
3 // FILE:   FinalProjectStarter_main.c
4 //
5 // TITLE:   Final Project Starter
6 //#####
7
8 // Included Files
9 #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15 #include "F28x_Project.h"
16 #include "F2837xD_SWPrioritizedIsrLevels.h"
17 #include "driverlib.h"
18 #include "device.h"
19 #include "F28379dSerial.h"
20 #include "song.h"
21 #include "dsp.h"
22 #include "fpu32/fpu_rfft.h"
23 #include "xy.h"
24 #include "MatrixMath.h"
25 #include "SE423Lib.h"
26 #include "OptiTrack.h"
27
28 #define PI          3.1415926535897932384626433832795
29 #define TWOPI       6.283185307179586476925286766559
30 #define HALFPi      1.5707963267948966192313216916398
31 // The Launchpad's CPU Frequency set to 200 you should not change this value
32 #define LAUNCHPAD_CPU_FREQUENCY 200
33
34 // ----- code for CAN start here -----
35 #include "F28379dCAN.h"
36 // #define TX_MSG_DATA_LENGTH      4
37 // #define TX_MSG_OBJ_ID          0 //transmit
38
39 #define RX_MSG_DATA_LENGTH      8
40 #define RX_MSG_OBJ_ID_1        1 //measurement from sensor 1
41 #define RX_MSG_OBJ_ID_2        2 //measurement from sensor 2
42 #define RX_MSG_OBJ_ID_3        3 //quality from sensor 1
43 #define RX_MSG_OBJ_ID_4        4 //quality from sensor 2
44 // ----- code for CAN end here -----
45
46 // Interrupt Service Routines predefinition
47 __interrupt void cpu_timer0_isr(void);
48 __interrupt void cpu_timer1_isr(void);
49 __interrupt void cpu_timer2_isr(void);
50 __interrupt void SWI1_HighestPriority(void);
51 __interrupt void SWI2_MiddlePriority(void);
52 __interrupt void SWI3_LowestPriority(void);
53 __interrupt void ADCC_ISR(void);
54 __interrupt void SPIB_isr(void);
55 // ----- code for CAN start here -----
56 __interrupt void can_isr(void);
57 // ----- code for CAN end here -----
58
59 void setF28027EPWM1A(float controleffort);
60 int16_t EPwm1A_F28027 = 1500;
61 void setF28027EPWM2A(float controleffort);
62 int16_t EPwm2A_F28027 = 1500;
63
64
65 // ----- code for CAN start here -----
66 // volatile uint32_t txMsgCount = 0;
67 // extern uint16_t txMsgData[4];
68
69 volatile uint32_t rxMsgCount_1 = 0;
70 volatile uint32_t rxMsgCount_2 = 0;
71 extern uint16_t rxMsgData[8];
72
73 uint32_t dis_raw_1[2];
74 uint32_t dis_raw_2[2];
75 uint32_t dis_1 = 0;
76 uint32_t dis_2 = 0;
77
78 uint32_t quality_raw_1[4];
79 uint32_t quality_raw_2[4];
80 float quality_1 = 0.0;
81 float quality_2 = 0.0;
82
83 uint32_t lightlevel_raw_1[4];
84 uint32_t lightlevel_raw_2[4];
85 float lightlevel_1 = 0.0;
86 float lightlevel_2 = 0.0;
87
88 uint32_t measure_status_1 = 0;
89 uint32_t measure_status_2 = 0;
90
91 volatile uint32_t errorFlag = 0;
92 // ----- code for CAN end here -----
93
94 uint32_t numTimer0calls = 0;
95 uint16_t UARTPrint = 0;
96
97 float printLV1 = 0;
98 float printLV2 = 0;
99
100 float printLinux1 = 0;
101 float printLinux2 = 0;
102
103 uint16_t LADARi = 0;
104 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
105 uint16_t G_len = 11; //length of command
106 xy ladar_pts[228]; //xy data
107 float LADARrightfront = 0;
108 float LADARfront = 0;
109 float LADARleftfront = 0;
110
111 float LADARtemp_x = 0;
112 float LADARtemp_y = 0;
113 extern datapts ladar_data[228];
114
115 extern uint16_t newLinuxCommands;
116 extern float LinuxCommands[CMDNUM_FROM_FLOATS];
117
118 extern uint16_t NewLVData;
119 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
120 extern LVSendFloats_t DataToLabView;
121 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
122 extern uint16_t LADARpingpong;
123 extern uint16_t NewCAMdataThreshold1; // Flag new data
124 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
125 extern uint16_t NewCAMdataThreshold2; // Flag new data
126 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
127 extern uint16_t NewCAMdataThreshold3; // Flag new data
128 extern float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
129
130 extern uint16_t NewCAMdataAprilTag1; // Flag new data
131 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
132
133 float tagid = 0;
134 float tagx = 0;
135 float tagy = 0;
136 float tagz = 0;
137 float tagthetax = 0;
138 float tagthetay = 0;
139 float tagthetaz = 0;
140 int32_t numtag = 0;

```

```

141 int32_t AprilTagState = 10;
142 float KpAprilTag = -1.0;
143 float AprilTagSetPoint = 0;
144 int32_t AprilTagCount = 0;
145
146 float MaxAreaThreshold1 = 0;
147 float MaxColThreshold1 = 0;
148 float MaxRowThreshold1 = 0;
149 float NextLargestAreaThreshold1 = 0;
150 float NextLargestColThreshold1 = 0;
151 float NextLargestRowThreshold1 = 0;
152 float NextNextLargestAreaThreshold1 = 0;
153 float NextNextLargestColThreshold1 = 0;
154 float NextNextLargestRowThreshold1 = 0;
155
156 float MaxAreaThreshold2 = 0;
157 float MaxColThreshold2 = 0;
158 float MaxRowThreshold2 = 0;
159 float NextLargestAreaThreshold2 = 0;
160 float NextLargestColThreshold2 = 0;
161 float NextLargestRowThreshold2 = 0;
162 float NextNextLargestAreaThreshold2 = 0;
163 float NextNextLargestColThreshold2 = 0;
164 float NextNextLargestRowThreshold2 = 0;
165
166 //EC
167 float MaxAreaThreshold3 = 0;
168 float MaxColThreshold3 = 0;
169 float MaxRowThreshold3 = 0;
170 float NextLargestAreaThreshold3 = 0;
171 float NextLargestColThreshold3 = 0;
172 float NextLargestRowThreshold3 = 0;
173 float NextNextLargestAreaThreshold3 = 0;
174 float NextNextLargestColThreshold3 = 0;
175 float NextNextLargestRowThreshold3 = 0;
176
177 uint32_t numThres1 = 0;
178 uint32_t numThres2 = 0;
179 uint32_t numThres3 = 0; //EC
180
181 pose ROBOTps = {0,0,0}; //robot position
182 pose LADARps = {3.5/12.0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
183 float LADARxoffset = 0;
184 float LADARyoffset = 0;
185
186 uint32_t timecount = 0;
187 int16_t RobotState = 1;
188 int32_t checkfronttally = 0;
189 int32_t WallFollowTime = 0;
190
191 #define NUMWAYPOINTS 4 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
192 uint16_t statePos = 0;
193 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
194 uint16_t i = 0; //for loop
195
196 uint16_t right_wall_follow_state = 2; // right follow
197 float Kp_front_wall_right = -2.0;
198 float front_turn_velocity = 0.2;
199
200 float Kp_right_wal = -4.0;
201 float ref_right_wall = 2.0;
202 float foward_velocity = 1.0;
203 float turn_saturation = 2.5;
204 float right_turn_Stop_threshold = 3.5;
205 float right_turn_Start_threshold = 1.3;
206
207
208 uint16_t left_wall_follow_state = 2; //DS: left wall follow state
209 float Kp_left_wal = 4.0; //DS: opposite sign of right wall follow
210 float ref_left_wall = 2.0; //DS: same desired wall distance as right wall
211 float left_turn_Stop_threshold = 3.5;
212 float left_turn_Start_threshold = 1.3;
213 float Kp_front_wall_left = 2.0;
214
215 //RC Servo
216 float Gate_0 = -60 ;
217 float Gate_C = 0 ;
218 float RTong = -49.34; //KLEC: ball goes to the left
219 float MTong = -11.49;
220 float LTong = 1.47; //KLEC: ball goes to the right
221 float dropOffStart = 0;
222
223 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
224
225 //more kalman vars
226 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
227 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
228 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
229 float z[3][1]; // state measurement
230 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
231 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
232 #define ProcUncert 0.0001
233 #define CovScalar 10
234 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
235 {0,ProcUncert,ProcUncert/CovScalar},
236 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
237 #define MeasUncert 1
238 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
239 {0,MeasUncert,MeasUncert/CovScalar},
240 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
241 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
242 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
243 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
244 float temp_3x3[3][3]; // intermediate storage
245 float temp_3x1[3][1]; // intermediate storage
246 float ytilde[3][1]; // difference between predictions
247
248 // Optitrack Variables
249 int32_t OptiNumUpdatesProcessed = 0;
250 pose OPTITRACKps;
251 extern uint16_t new_optitrack;
252 extern float Optitrackdata[OPTITRACKDATASIZE];
253 int16_t newOPTITRACKpose=0;
254
255 int16_t adcc2result = 0;
256 int16_t adcc3result = 0;
257 int16_t adcc4result = 0;
258 int16_t adcc5result = 0;
259 float adcc2Volt = 0.0;
260 float adcc3Volt = 0.0;
261 float adcc4Volt = 0.0;
262 float adcc5Volt = 0.0;
263 int32_t numADCCcalls = 0;
264 float LeftWheel = 0;
265 float RightWheel = 0;
266 float LeftWheel_1 = 0;
267 float RightWheel_1 = 0;
268 float LeftVel = 0;
269 float RightVel = 0;
270 float uLeft = 0;
271 float uRight = 0;
272 float HandValue = 0;
273 float vref = 0;
274 float turn = 0;
275 float gyro9250_drift = 0;
276 float gyro9250_angle = 0;
277 float old_gyro9250 = 0;
278 float gyroLPRS10_angle = 0;
279 float gyroLPRS10_offset = 0;
280 float gyroLPRS10_drift = 0;
281 float old_gyroLPRS10 = 0;
282 float gyro9250_radians = 0;

```

```

283 float gyroLPR510_radians = 0;
284
285 int16_t readdata[25];
286 int16_t IMU_data[9];
287
288 float accelx = 0;
289 float accely = 0;
290 float accelz = 0;
291 float gyrox = 0;
292 float gyroy = 0;
293 float gyroz = 0;
294
295 // Needed global Variables
296 float accelx_offset = 0;
297 float accely_offset = 0;
298 float accelz_offset = 0;
299 float gyrox_offset = 0;
300 float gyroy_offset = 0;
301 float gyroz_offset = 0;
302 int16_t calibration_state = 0;
303 int32_t calibration_count = 0;
304 int16_t doneCal = 0;
305
306 #define MPU9250 1
307 #define DAN28027 2
308 int16_t CurrentChip = MPU9250;
309 int16_t DAN28027Garbage = 0;
310 int16_t dan28027adc1 = 0;
311 int16_t dan28027adc2 = 0;
312 uint16_t MPU9250IgnoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
313 //Added variables to store camera-based ball distances - DS
314 float robotToBall2 = 0;
315 float robotToBall1 = 0;
316 float robotToBall3 = 0;
317
318 // angle of the hand encoder - ER
319 float RAngle = 0.0;
320 float colcentroid1 = 0.0;
321 float colcentroid2 = 0.0;
322 float colcentroid3 = 0.0;
323 float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS
324
325 int32_t count = 0;
326 int32_t statecount = 2000;
327 int16_t dwell=0;
328 float current= 0.0;
329 float target = 0.0;
330
331 void main(void)
332 {
333     // PLL, WatchDog, enable Peripheral Clocks
334     // This example function is found in the F2837xD_SysCtrl.c file.
335     InitSysCtrl();
336
337     InitGpio();
338
339     InitSE423DefaultGPIO();
340
341     //Scope for Timing
342     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
343     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
344     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
345
346     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
347     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
348     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
349
350     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
351     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
352     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
353
354     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
355     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
356     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
357
358     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
359     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
360     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
361
362     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
363     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
364     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
365
366     // ----- code for CAN start here -----
367     //GPIO17 - CANRXB
368     GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
369     GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
370
371     //GPIO12 - CANTXB
372     GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
373     GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
374     // ----- code for CAN end here -----
375
376
377     // ----- code for CAN start here -----
378     // Initialize the CAN controller
379     InitCANB();
380
381     // Set up the CAN bus bit rate to 1000 kbps
382     setCANBitRate(200000000, 1000000);
383
384     // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
385     CanbRegs.CAN_CTL.bit.IE0 = 1;
386     CanbRegs.CAN_CTL.bit.EIE = 1;
387     // ----- code for CAN end here -----
388
389     // Clear all interrupts and initialize PIE vector table:
390     // Disable CPU interrupts
391     DINT;
392
393     // Initialize the PIE control registers to their default state.
394     // The default state is all PIE interrupts disabled and flags
395     // are cleared.
396     // This function is found in the F2837xD_PieCtrl.c file.
397     InitPieCtrl();
398
399     // Disable CPU interrupts and clear all CPU interrupt flags:
400     IER = 0x0000;
401     IFR = 0x0000;
402
403     // Initialize the PIE vector table with pointers to the shell Interrupt
404     // Service Routines (ISR).
405     // This will populate the entire table, even if the interrupt
406     // is not used in this example. This is useful for debug purposes.
407     // The shell ISR routines are found in F2837xD_DefaultIsr.c.
408     // This function is found in F2837xD_PieVect.c.
409     InitPieVectTable();
410
411     // Interrupts that are used in this example are re-mapped to
412     // ISR functions found within this project
413     EALLOW; // This is needed to write to EALLOW protected registers
414     PieVectTable.TIMER0_INT = &cpu_timer0_isr;
415     PieVectTable.TIMER1_INT = &cpu_timer1_isr;
416     PieVectTable.TIMER2_INT = &cpu_timer2_isr;
417     PieVectTable.SCIA_RX_INT = &RXAINT_rcv_ready;
418     PieVectTable.SCIB_RX_INT = &RXBINT_rcv_ready;
419     PieVectTable.SCIC_RX_INT = &RXCINT_rcv_ready;
420     PieVectTable.SCID_RX_INT = &RXDINT_rcv_ready;
421     PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
422     PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
423     PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;

```

```

425 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
426 PieVectTable.ADCC1_INT = &ADCC_ISR;
427 PieVectTable.SPIB_RX_INT = &SPIB_isr;
428 PieVectTable.ENTF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
429 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
430 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
431 // ----- code for CAN start here -----
432 PieVectTable.CANB0_INT = &can_isr;
433 // ----- code for CAN end here -----
434 EDIS; // This is needed to disable write to EALLOW protected registers
435
436 // Initialize the CpuTimers Device Peripheral. This function can be
437 // found in F2837xD_CpuTimers.c
438 InitCpuTimers();
439
440 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
441 // 200MHz CPU Freq, Period (in uSeconds)
442 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
443 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
444 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
445
446 // Enable CpuTimer Interrupt bit TIE
447 CpuTimer0Regs.TCR.all = 0x4000;
448 CpuTimer1Regs.TCR.all = 0x4000;
449 CpuTimer2Regs.TCR.all = 0x4000;
450
451 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
452
453 init_serialSCIB(&SerialB,19200);
454 init_serialSCIC(&SerialC,19200);
455
456 for (LADARi = 0; LADARi < 228; LADARi++) {
457     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
458 }
459
460 init_eQEPs();
461 init_EPWM1and2();
462 init_RCServoPWM_3AB_5AB_6A();
463 init_ADCsAndDACs();
464 setupSpib();
465
466 EALLOW;
467 EPwm4Regs.ETSEL.bit.SOCACEN = 0; // Disable SOC on A group
468 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
469 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
470 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
471 EPwm4Regs.TBCTR = 0x0; // Clear counter
472 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
473 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
474 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
475 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
476 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
477 EPwm4Regs.ETSEL.bit.SOCACEN = 1; //enable SOCA
478 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
479 EDIS;
480
481 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
482 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
483 // to CPU-Timer 2: int 12 is for the SWI.
484 IER |= M_INT1;
485 IER |= M_INT6;
486 IER |= M_INT8; // SCIC SCID
487 IER |= M_INT9; // SCIA
488 IER |= M_INT12;
489 IER |= M_INT13;
490 IER |= M_INT14;
491
492 // Enable TINT0 in the PIE: Group 1 interrupt 7
493 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
494 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
495 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
496 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
497 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
498 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
499 // ----- code for CAN start here -----
500 // Enable CANB in the PIE: Group 9 interrupt 7
501 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
502 // ----- code for CAN end here -----
503
504 // ----- code for CAN start here -----
505 // Enable the CAN interrupt signal
506 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
507 // ----- code for CAN end here -----
508
509 // 58.5 in * 7.5 tiles = 438.75 inches = 11.14425 meters
510
511
512 robotdest[0].x = 0; robotdest[0].y = 8*5;
513 robotdest[1].x = -8*5; robotdest[1].y = 8*5;
514 //middle of bottom
515 robotdest[2].x = 0; robotdest[2].y = 8*5;
516 // //outside the course
517 robotdest[3].x = 0; robotdest[3].y = 0;
518 // //back to middle
519 // robotdest[4].x = 0; robotdest[4].y = 2;
520 // robotdest[5].x = 4; robotdest[5].y = 2;
521 // robotdest[6].x = 4; robotdest[6].y = 10;
522 // robotdest[7].x = 0; robotdest[7].y = 9;
523 // // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
524 // robotdest[8].x = 2; robotdest[8].y = 5;
525 // robotdest[9].x = -2; robotdest[9].y = -1;
526
527 // ROBOTps will be updated by Optitrack during gyro calibration
528 // TODO: specify the starting position of the robot
529 ROBOTps.x = 0; //the estimate in array form (useful for matrix operations)
530 ROBOTps.y = 0;
531 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
532 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
533 x_pred[1][0] = ROBOTps.y;
534 x_pred[2][0] = ROBOTps.theta;
535
536 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
537 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
538 SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag
539 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
540 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
541 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
542
543 init_serialSCIA(&SerialA,115200);
544 init_serialSCID(&SerialD,2083332);
545
546 // Enable global Interrupts and higher priority real-time debug events
547 EINT; // Enable Global interrupt INTM
548 ERTM; // Enable Global realtime interrupt DBGM
549 // ----- code for CAN start here -----
550 // Measured Distance from 1
551 // Initialize the receive message object 1 used for receiving CAN messages.
552 // Message Object Parameters:
553 // Message Object ID Number: 1
554 // Message Identifier: 0x060b0101
555 // Message Frame: Standard
556 // Message Type: Receive
557 // Message ID Mask: 0x0
558 // Message Object Flags: Receive Interrupt
559 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
560 // for a Receive mailbox)
561 //
562 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
563 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
564 RX_MSG_DATA_LENGTH);
565
566 // Measured Distance from 2

```

```

567 // Initialize the receive message object 2 used for receiving CAN messages.
568 // Message Object Parameters:
569 //     Message Object ID Number: 2
570 //     Message Identifier: 0x060b0102
571 //     Message Frame: Standard
572 //     Message Type: Receive
573 //     Message ID Mask: 0x0
574 //     Message Object Flags: Receive Interrupt
575 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
576 //     for a Receive mailbox)
577 //
578
579 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
580                      CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
581                      RX_MSG_DATA_LENGTH);
582
583 // Measurement Quality from 1
584 // Initialize the receive message object 2 used for receiving CAN messages.
585 // Message Object Parameters:
586 //     Message Object ID Number: 3
587 //     Message Identifier: 0x060b0201
588 //     Message Frame: Standard
589 //     Message Type: Receive
590 //     Message ID Mask: 0x0
591 //     Message Object Flags: Receive Interrupt
592 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
593 //     for a Receive mailbox)
594 //
595
596 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
597                      CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
598                      RX_MSG_DATA_LENGTH);
599
600 // Measurement Quality from 2
601 // Initialize the receive message object 2 used for receiving CAN messages.
602 // Message Object Parameters:
603 //     Message Object ID Number: 4
604 //     Message Identifier: 0x060b0202
605 //     Message Frame: Standard
606 //     Message Type: Receive
607 //     Message ID Mask: 0x0
608 //     Message Object Flags: Receive Interrupt
609 //     Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
610 //     for a Receive mailbox)
611 //
612
613 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
614                      CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
615                      RX_MSG_DATA_LENGTH);
616
617 //
618 // Start CAN module operations
619 //
620 CanbRegs.CAN_CTL.bit.Init = 0;
621 CanbRegs.CAN_CTL.bit.CCE = 0;
622
623
624 // ----- code for CAN end here -----
625 char S_command[19] = "S1152000124000\n";//this change the baud rate to 115200
626 uint16_t S_len = 19;
627 serial_sendSCIC(&SerialC, S_command, S_len);
628
629
630 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
631 init_serialSCIC(&SerialC, 115200);
632
633
634 // LED1 is GPIO22
635 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
636 // LED2 is GPIO94
637 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
638 // LED3 is GPIO95
639 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
640 // LED4 is GPIO97
641 GpioDataRegs.GPCCLEAR.bit.GPIO97 = 1;
642 // LED5 is GPIO111
643 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
644
645
646 // IDLE loop. Just sit and loop forever (optional):
647 while(1)
648 {
649     if (UARTPrint == 1) {
650         //UART_printfLine(1,"RCangle: %.2f", RCangle);
651         if (readbuttons() == 0) {
652             UART_printfLine(1,"RS: %d TC %ld", RobotState, checkfrontally);
653             //             UART_printfLine(1,"O1A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
654             //             UART_printfLine(2,"PIA: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
655             UART_printfLine(2, "LF %.2f SC %d", LADARfront, statecount);
656             //             UART_printfLine(1,"x: %.2f: y: %.2f: a: %.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
657             //             UART_printfLine(2,"purple: %.2f", robotToBall2);
658         } else if (readbuttons() == 1) {
659             //             UART_printfLine(1,"O1A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
660             //             UART_printfLine(2,"PIA: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
661             //UART_printfLine(1,"LV1: %.3f LV2: %.3f", printLV1, printLV2);
662             //UART_printfLine(2,"Ln1: %.3f Ln2: %.3f", printLinux1, printLinux2);
663         } else if (readbuttons() == 2) {
664             UART_printfLine(1,"O2A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
665             UART_printfLine(2,"P2A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
666             // UART_printfLine(1,"%.2f %.2f", adcC2Volt, adcC3Volt);
667             // UART_printfLine(2,"%.2f %.2f", adcC4Volt, adcC5Volt);
668         } else if (readbuttons() == 4) {
669             UART_printfLine(1,"O3A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
670             UART_printfLine(2,"P3A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
671             // UART_printfLine(1,"uL: %.3f R: %.3f", LeftVel, RightVel);
672             // UART_printfLine(2,"uL: %.2f uR: %.2f", uLeft, uRight);
673         } else if (readbuttons() == 8) {
674             UART_printfLine(1,"O20x: %.2f y: %.2f", ladar_pts[20].x, ladar_pts[20].y);
675             UART_printfLine(2,"I50x: %.2f y: %.2f", ladar_pts[150].x, ladar_pts[150].y);
676         } else if (readbuttons() == 3) {
677             UART_printfLine(1,"Vrf: %.2f trn: %.2f", vref, turn);
678             UART_printfLine(2,"MPU: %.2f LPR: %.2f", gyro9250_radians, gyroLPR510_radians);
679         } else if (readbuttons() == 5) {
680             UART_printfLine(1,"Ox: %.2f Oy: %.2f: Oa: %.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
681             UART_printfLine(2,"State: %d : %d", RobotState, statePos);
682         } else if (readbuttons() == 6) {
683             UART_printfLine(1,"D1 %ld D2 %ld", dis_1, dis_2);
684             UART_printfLine(2,"St1 %ld St2 %ld", measure_status_1, measure_status_2);
685         } else if (readbuttons() == 7) {
686             UART_printfLine(1,"%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
687             UART_printfLine(2,"%.1f, %.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
688         }
689     }
690     UARTPrint = 0;
691 }
692 }
693 }
694
695 __interrupt void SPIB_isr(void){
696
697     uint16_t i;
698     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
699
700     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
701     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
702
703     if (CurrentChip == MPU9250) {
704
705         for (i=0; i<8; i++) {
706             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
707         }
708     }

```

```

709     PostSWI1(); // Manually cause the interrupt for the SWI1
710 } else if (CurrentChip == DAN28027) {
711     DAN28027Garbage = SpibRegs.SPIRXBUF;
712     dan28027adc1 = SpibRegs.SPIRXBUF;
713     dan28027adc2 = SpibRegs.SPIRXBUF;
714     CurrentChip = MPU9250;
715     SpibRegs.SPIFFCT.bit.TXDLY = 0;
716 }
717
718 SpibRegs.SPIFFRX.bit.RXFFOVFLR=1; // Clear Overflow flag
719 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
720 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
721 GpioDataRegs.GPBCLLEAR.bit.GPIO32 = 1;
722 }
723
724 //adc1 pie interrupt
725 __interrupt void ADCC_ISR (void)
726 {
727     GpioDataRegs.GPASET.bit.GPIO11 = 1;
728     adcc2result = AdccResultRegs.ADCRESULT0;
729     adcc3result = AdccResultRegs.ADCRESULT1;
730     adcc4result = AdccResultRegs.ADCRESULT2;
731     adcc5result = AdccResultRegs.ADCRESULT3;
732
733     // Here covert ADCIND0 to volts
734     adcC2Volt = adcc2result*3.0/4095.0;
735     adcC3Volt = adcc3result*3.0/4095.0;
736     adcC4Volt = adcc4result*3.0/4095.0;
737     adcC5Volt = adcc5result*3.0/4095.0;
738
739     if (MPU9250ignoreCNT >= 1) {
740         CurrentChip = MPU9250;
741         SpibRegs.SPIFFCT.bit.TXDLY = 0;
742         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
743
744         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
745
746         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
747         SpibRegs.SPITXBUF = 0;
748         SpibRegs.SPITXBUF = 0;
749         SpibRegs.SPITXBUF = 0;
750         SpibRegs.SPITXBUF = 0;
751         SpibRegs.SPITXBUF = 0;
752         SpibRegs.SPITXBUF = 0;
753         SpibRegs.SPITXBUF = 0;
754         SpibRegs.SPITXBUF = 0;
755         SpibRegs.SPITXBUF = 0;
756         SpibRegs.SPITXBUF = 0;
757     } else {
758         MPU9250ignoreCNT++;
759     }
760
761     numADCCcalls++;
762     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
763     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
764     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
765 }
766
767 // cpu_timer0_isr - CPU Timer0 ISR
768 __interrupt void cpu_timer0_isr(void)
769 {
770     CpuTimer0.InterruptCount++;
771
772     numTimer0calls++;
773
774     if ((numTimer0calls%5) == 0) {
775         // Blink LaunchPad Red LED
776         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
777     }
778
779     // Acknowledge this interrupt to receive more interrupts from group 1
780     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
781 }
782
783 // cpu_timer1_isr - CPU Timer1 ISR
784 __interrupt void cpu_timer1_isr(void)
785 {
786     serial_sendSCIC(&SerialC, G_command, G_len);
787
788     CpuTimer1.InterruptCount++;
789 }
790
791 // cpu_timer2_isr CPU Timer2 ISR
792 __interrupt void cpu_timer2_isr(void)
793 {
794     // Blink LaunchPad Blue LED
795     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
796     PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
797     CpuTimer2.InterruptCount++;
798
799     // if ((CpuTimer2.InterruptCount % 10) == 0) {
800     //     UARTPrint = 1;
801     // }
802
803 void setF28027EPWM1A(float controleffort){
804     if (controleffort < -10) {
805         controleffort = -10;
806     }
807     if (controleffort > 10) {
808         controleffort = 10;
809     }
810     float value = (controleffort+10)*3000.0/20.0;
811     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
812 }
813 void setF28027EPWM2A(float controleffort){
814     if (controleffort < -10) {
815         controleffort = -10;
816     }
817     if (controleffort > 10) {
818         controleffort = 10;
819     }
820     float value = (controleffort+10)*3000.0/20.0;
821     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
822 }
823
824 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
825 //
826 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
827 {
828     GpioDataRegs.GPBSET.bit.GPIO61 = 1;
829     // Set interrupt priority;
830     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
831     IER |= M_INT12;
832     IER &= MINT12; // Set "global" priority
833     PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
834     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
835     _asm(" NOP");
836     EINT;
837
838     //=====
839     IMU_data[0] = readdata[1];
840     IMU_data[1] = readdata[2];
841     IMU_data[2] = readdata[3];
842     IMU_data[3] = readdata[5];
843     IMU_data[4] = readdata[6];
844     IMU_data[5] = readdata[7];
845
846     accelx = (((float)(IMU_data[0]))*4.0/32767.0);

```

```

851 accelx = (((float)(IMU_data[1]))*4.0/32767.0);
852 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
853 gyroX = (((float)(IMU_data[3]))*250.0/32767.0);
854 gyroY = (((float)(IMU_data[4]))*250.0/32767.0);
855 gyroZ = (((float)(IMU_data[5]))*250.0/32767.0);
856
857 if(calibration_state == 0){
858     calibration_count++;
859     if (calibration_count == 2000) {
860         calibration_state = 1;
861         calibration_count = 0;
862     }
863 } else if(calibration_state == 1){
864     accelx_offset+=accelx;
865     accely_offset+=accely;
866     accelz_offset+=accelz;
867     gyroX_offset+=gyroX;
868     gyroY_offset+=gyroY;
869     gyroZ_offset+=gyroZ;
870     gyroLPR510_offset+=adcC5Volt;
871     calibration_count++;
872     if (calibration_count == 2000) {
873         calibration_state = 2;
874         accelx_offset/=2000.0;
875         accely_offset/=2000.0;
876         accelz_offset/=2000.0;
877         gyroX_offset/=2000.0;
878         gyroY_offset/=2000.0;
879         gyroZ_offset/=2000.0;
880         gyroLPR510_offset/=2000.0;
881         calibration_count = 0;
882     }
883 } else if(calibration_state == 2) {
884
885     gyroLPR510_drift += ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
886     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
887
888     gyro9250_drift += (((gyroZ-gyroZ_offset) + old_gyro9250)*.0005)/(2000);
889     old_gyro9250 = gyroZ-gyroZ_offset;
890     calibration_count++;
891
892     if (calibration_count == 2000) {
893         calibration_state = 3;
894         calibration_count = 0;
895         doneCal = 1;
896         newOPTITRACKpose = 0;
897     }
898 } else if(calibration_state == 3){
899     accelx -=(accelx_offset);
900     accely -=(accely_offset);
901     accelz -=(accelz_offset);
902     gyroX -= gyroX_offset;
903     gyroY -= gyroY_offset;
904     gyroZ -= gyroZ_offset;
905     LeftWheel = readEncLeft();
906     RightWheel = readEncRight();
907     HandValue = readEncWheel();
908
909     gyro9250_angle = gyro9250_angle + (gyroZ + old_gyro9250)*.0005 - gyro9250_drift;
910     old_gyro9250 = gyroZ;
911
912     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
913     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
914
915     gyro9250_radians = (gyro9250_angle * (PI/180.0));
916     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
917
918     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
919     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
920
921     if (newLinuxCommands == 1) {
922         newLinuxCommands = 0;
923         printLinux1 = LinuxCommands[0];
924         printLinux2 = LinuxCommands[1];
925         //?? = LinuxCommands[2];
926         //?? = LinuxCommands[3];
927         //?? = LinuxCommands[4];
928         //?? = LinuxCommands[5];
929         //?? = LinuxCommands[6];
930         //?? = LinuxCommands[7];
931         //?? = LinuxCommands[8];
932         //?? = LinuxCommands[9];
933         //?? = LinuxCommands[10];
934     }
935
936     if (NewCAMDataThreshold1 == 1) {
937         NewCAMDataThreshold1 = 0;
938         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
939         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
940         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
941
942         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
943         robotToBall1 = -0.00002154*MaxRowThreshold1*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;
944
945         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
946         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
947         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
948
949         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
950         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
951         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
952         numThres1++;
953         if ((numThres1 % 5) == 0) {
954             // LED4 is GPIO97
955             GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
956         }
957     }
958
959     if (NewCAMDataThreshold2 == 1) {
960         NewCAMDataThreshold2 = 0;
961         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
962         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
963         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
964         // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
965
966         if (ROBOTps.y < 35) { //first hallway
967             robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
968         }
969         else if (ROBOTps.y >= 35) { //second hallway
970             if (MaxRowThreshold2 > 40)
971                 robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;
972             else if (MaxRowThreshold2 <= 40)
973                 robotToBall2 = 0;
974         }
975         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
976         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
977         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
978
979         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
980         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
981         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
982         numThres2++;
983         if ((numThres2 % 5) == 0) {
984             // LED5 is GPIO111
985             GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
986         }
987     }
988     if (NewCAMDataThreshold3 == 1) {
989         NewCAMDataThreshold3 = 0;
990         MaxAreaThreshold3 = fromCAMvaluesThreshold3[0];
991         MaxColThreshold3 = fromCAMvaluesThreshold3[1];
992         MaxRowThreshold3 = fromCAMvaluesThreshold3[2];

```



```

993 // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
994 robotToBall3 = -0.00002154*MaxRowThresholD3*MaxRowThresholD3*MaxRowThresholD3 + 0.006486*MaxRowThresholD3*MaxRowThresholD3 - 0.6728*MaxRowThresholD3 + 25.42;
995
996 NextLargestAreaThreshold3 = fromCAMvaluesThreshold3[3];
997 NextLargestColThreshold3 = fromCAMvaluesThreshold3[4];
998 NextLargestRowThreshold3 = fromCAMvaluesThreshold3[5];
999
1000 NextNextLargestAreaThreshold3 = fromCAMvaluesThreshold3[6];
1001 NextNextLargestColThreshold3 = fromCAMvaluesThreshold3[7];
1002 NextNextLargestRowThreshold3 = fromCAMvaluesThreshold3[8];
1003 numThres3++;
1004 if ((numThres3 % 5) == 0) {
1005     // LED5 is GPIO111
1006     GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
1007 }
1008
1009
1010 if (NewCAMDataAprilTag1 == 1) {
1011     NewCAMDataAprilTag1 = 0;
1012     tagx = fromCAMvaluesAprilTag1[0];
1013     tagy = fromCAMvaluesAprilTag1[1];
1014     tagz = fromCAMvaluesAprilTag1[2];
1015
1016     tagthetax = fromCAMvaluesAprilTag1[3];
1017     tagthetay = fromCAMvaluesAprilTag1[4];
1018     tagthetaz = fromCAMvaluesAprilTag1[5];
1019
1020     tagid = fromCAMvaluesAprilTag1[6];
1021
1022     numtag++;
1023     if ((numtag % 5) == 0) {
1024         // LED2 is GPIO94
1025         GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
1026     }
1027 }
1028
1029 if (NewLVData == 1) {
1030     NewLVData = 0;
1031     printLV1 = fromLVvalues[0];
1032     printLV2 = fromLVvalues[1];
1033     //?? = fromLVvalues[2];
1034     //?? = fromLVvalues[3];
1035     //?? = fromLVvalues[4];
1036     //?? = fromLVvalues[5];
1037     //?? = fromLVvalues[6];
1038     //?? = fromLVvalues[7];
1039 }
1040
1041
1042 if (new_optitrack == 1) {
1043     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
1044     new_optitrack = 0;
1045 }
1046
1047 // Step 0: update B, u
1048 B[0][0] = cosf(ROBOTps.theta)*0.001;
1049 B[1][0] = sinf(ROBOTps.theta)*0.001;
1050 B[2][1] = 0.001;
1051 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
1052 u[1][0] = gyroZ*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
1053
1054 // Step 1: predict the state and estimate covariance
1055 Matrix3x2_Mult(B, u, Bu); // Bu = B*u
1056 Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
1057 Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1058 // Step 2: if there is a new measurement, then update the state
1059 if (1 == newOPTITRACKpose) {
1060     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1061     if ((OptiNumUpdatesProcessed%10)==0) {
1062         // LED 3
1063         GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
1064     }
1065     newOPTITRACKpose = 0;
1066     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1067     z[1][0] = OPTITRACKps.y;
1068     // fix for OptiTrack problem at 180 degrees
1069     if (cosf(ROBOTps.theta) < -0.99) {
1070         z[2][0] = ROBOTps.theta;
1071     }
1072     else {
1073         z[2][0] = OPTITRACKps.theta;
1074     }
1075     // Step 2a: calculate the innovation/measurement residual, ytilde
1076     Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1077     // Step 2b: calculate innovation covariance, S
1078     Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1079     // Step 2c: calculate the optimal Kalman gain, K
1080     Matrix3x3_Invert(S, S_inv);
1081     Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1082     // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1083     Matrix3x1_Mult(K, ytilde, temp_3x1);
1084     Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1085     // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1086     Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1087     Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1088 } // end of correction step
1089
1090 // set ROBOTps to the updated and corrected Kalman values.
1091 ROBOTps.x = x_pred[0][0];
1092 ROBOTps.y = x_pred[1][0];
1093 ROBOTps.theta = x_pred[2][0];
1094
1095 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1096 // uses xy code to step through an array of positions
1097 if (xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1098     statePos = (statePos+1)%NUMWAYPOINTS;
1099 }
1100
1101 }
1102
1103 dwell++;
1104 // state machine
1105 colcentroid1 = MaxColThreshold1 - 80;
1106 colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives centroid = 0 - DS
1107 colcentriod3 = MaxColThreshold3 - 80;
1108 switch (RobotState) {
1109     case 1: // command the robot to an X,Y point in the course - KL
1110         // vref and turn are the vref and turn returned from xy_control
1111         if (LADARfront < 1.2) {
1112             vref = 0.2;
1113             checkfronttally++;
1114             //DS: choose the side with more open space for obstacle avoidance
1115             if (checkfronttally > 500) {
1116                 checkfronttally = 501; // no overflow error
1117                 WallFollowtime = 0;
1118                 if (LADARrightfront < LADARleftfront) {
1119                     RobotState = 10; //DS: use right-wall following
1120                     right_wall_follow_state = 1;
1121                 } else {
1122                     RobotState = 12; //DS: use left-wall following
1123                     left_wall_follow_state = 1;
1124                 }
1125             }
1126         } else {
1127             checkfronttally = 0;
1128         }
1129         // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
1130         statecount++;
1131         if (statecount<=2000){
1132             RobotState = 1;
1133         }else{
1134             statecount = 2001; // No overflow error

```



```

1135     count = 0;
1136     if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
1137         RobotState = 20;
1138         statecount = 0;
1139     }
1140     if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
1141         RobotState = 30;
1142         statecount = 0;
1143     }
1144     if (fabs(tagz) <= 350.468262) {
1145         if (tagid == 0.0){
1146             RobotState = 99; //40;
1147             count=0;
1148         }
1149         if (tagid == 1.0){
1150             RobotState = 50;
1151             count=0;
1152         }
1153     }
1154 }
1155 break;
1156
1157 case 10: // right wall following if an obstacle gets in the way of robot - KL
1158 if (right_wall_follow_state == 1) {
1159     //Left Turn
1160     turn = Kp_front_wall_right*(14.5 - LADARfront);
1161     vref = front_turn_velocity;
1162     if (LADARfront > left_turn_Stop_threshold) {
1163         right_wall_follow_state = 2;
1164     }
1165 } else if (right_wall_follow_state == 2) {
1166     //Right Wall Follow
1167     turn = Kp_right_wall*(ref_right_wall - LADARrightfront);
1168     vref = foward_velocity;
1169     if (LADARfront < left_turn_Start_threshold) {
1170         right_wall_follow_state = 1;
1171     }
1172 }
1173 if (turn > turn_saturation) {
1174     turn = turn_saturation;
1175 }
1176 if (turn < -turn_saturation) {
1177     turn = -turn_saturation;
1178 }
1179
1180 WallFollowtime++;
1181 // exit wall following if at least 5 seconds have passed and front path is clear - KL
1182 if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1183     RobotState = 1; //return to XY waypoint navigation - KL
1184     checkfronttally = 0;
1185 } else if (ROBOTps.x == 0 && ROBOTps.y == 0.5){
1186     RobotState = 88;
1187     count = 0;
1188 }
1189 break;
1190
1191 case 12: //DS: left wall following if an obstacle gets in the way of robot
1192 if (left_wall_follow_state == 1) {
1193     //DS: Right Turn for left-wall following
1194     turn = Kp_front_wall_left*(14.5 - LADARfront);
1195     vref = front_turn_velocity;
1196     if (LADARfront > right_turn_Stop_threshold) {
1197         left_wall_follow_state = 2;
1198     }
1199 } else if (left_wall_follow_state == 2) {
1200     //DS: Left Wall Follow
1201     turn = Kp_left_wall*(ref_left_wall - LADARleftfront);
1202     vref = foward_velocity;
1203     if (LADARfront < right_turn_Start_threshold) {
1204         left_wall_follow_state = 1;
1205     }
1206 }
1207
1208 if (turn > turn_saturation) {
1209     turn = turn_saturation;
1210 }
1211 if (turn < -turn_saturation) {
1212     turn = -turn_saturation;
1213 }
1214
1215 WallFollowtime++;
1216 //DS: exit left-wall following if at least 5 seconds have passed and front path is clear
1217 if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1218     RobotState = 1; //DS: return to XY waypoint navigation
1219     checkfronttally = 0;
1220 }
1221 break;
1222 // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
1223 case 20:
1224     // put vision code here
1225     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
1226         vref = 0;
1227         turn = 0;
1228     } else {
1229         vref = 0.75;
1230         turn = kpvision * (0 - colcentroid1);
1231         if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
1232             RobotState = 22;
1233             count = 0;
1234         }
1235     }
1236 }
1237 break;
1238
1239 case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
1240 vref = 0;
1241 turn = 0;
1242
1243 count++;
1244 if (count>=1000){//after 1 second, robot drives forward - KL
1245     RobotState = 24;
1246     count = 0;
1247 }
1248 break;
1249
1250 case 24: // robot drives forward into the golf ball - KL
1251 vref = 0.5;
1252 turn = 0;
1253 setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
1254 setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1255 count++;
1256 if (count>=2000){
1257     RobotState = 26;
1258     count = 0;
1259 }
1260 }
1261 break;
1262
1263 case 26: // robot pauses once ball is presumed to be captured - KL
1264 vref = 0;
1265 turn = 0;
1266 setEPWM6A_RCServo(Gate_C);
1267 setEPWM5B_RCServo(MTong);
1268 count++;
1269 if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
1270     RobotState = 1;
1271     count = 0;
1272     statecount = 0;
1273 }
1274 }
1275 break;
1276
1277 // states for orange ball
1278 case 30:
1279     // put vision code here
1280     //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
1281     //this helps it not detect other orange objects in the room as the ball - KP
1282     if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
1283         vref = 0;

```

```

1277         turn = 0;
1278     } else {
1279         //tells it what to do if it is detected, move to state 32 - KP
1280         vref = 0.75;
1281         turn = kpvision * (0 - colcentroid2);
1282         if (MaxRowThreshold2 > 100){
1283             RobotState = 32;
1284             count = 0;
1285         }
1286     }
1287     break;
1288 case 32:
1289     //stopping and pausing with vref and turn at 0 - KP
1290     vref = 0;
1291     turn = 0;
1292     setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
1293     setEPWM6A_RCServo(Gate_0); //KLEC: open the gate
1294     count++;
1295     if (count>=1000){
1296         RobotState = 34;
1297         count = 0;
1298     }
1299     break;
1300 case 34:
1301     //telling robot to move forward then move to state 36 - KP
1302     vref = 0.5;
1303     turn = 0;
1304     setEPWM5B_RCServo(RTong);
1305     setEPWM6A_RCServo(Gate_0);
1306     count++;
1307     if (count>=1000){
1308         RobotState = 36;
1309         count = 0;
1310     }
1311     break;
1312 case 36:
1313     //another stop before it begins the route again - KP
1314     vref = 0;
1315     turn = 0;
1316     setEPWM6A_RCServo(Gate_C);
1317     setEPWM5B_RCServo(MTong);
1318     count++;
1319     if (count>=1000){
1320         RobotState = 1;
1321         count = 0;
1322         statecount = 0;
1323     }
1324     break;
1325     // case 40://code telling the gate to open when it detects april tag 0
1326     // vref = 0;
1327     // turn = 0;
1328     //
1329     // setEPWM6A_RCServo(Gate_0);
1330     // setEPWM5B_RCServo(0);
1331     //
1332     // if (dwell >= 2000){
1333     //     setEPWM6A_RCServo(Gate_C);
1334     // }
1335     //
1336     // count++;
1337     // if (count>=1000){
1338     //     RobotState = 1;
1339     //     count = 0;
1340     //     statecount = 0;
1341     // }
1342     // break;
1343 case 50: //code telling the robot to turn
1344     vref = 0;
1345     count++;
1346     if (count>=2000){
1347         target = ROBOTps.theta + PI;
1348         RobotState = 52;
1349         count = 0;
1350     }
1351     break;
1352 case 52:
1353     turn = -(target - ROBOTps.theta);
1354     vref = 0;
1355     count++;
1356     if (fabs(target - ROBOTps.theta)<(PI/6)){
1357         statePos = 3;
1358         RobotState = 1;
1359         count = 0;
1360         statecount = 0;
1361         vref = 1.5;
1362     }
1363     break;
1364 case 99: //robot dispatching the ball // Entering condition: got back to the home April tag location
1365     count++;
1366     vref = 0;
1367     turn = 0;
1368     //Drop off the color 1 green
1369     if (count <= 2000){
1370         vref = 0;
1371         turn = 0;
1372     } else if (count <= 4000){
1373         setEPWM5B_RCServo(LTong); //EC: Tongue move to left location
1374         setEPWM6A_RCServo(Gate_0); //Lift arm
1375         target = ROBOTps.theta + PI;
1376     } else if (count <= 7000){ //Back up fast for 3 seconds
1377         vref = -0.5;
1378     } else if (count <= 11000){
1379         vref = 0;
1380         setEPWM5B_RCServo(RTong); //Tongue right
1381         setEPWM6A_RCServo(Gate_C);
1382         turn = -(target - ROBOTps.theta);
1383     } else if(count <= 13000){ //Drop off the color 2 - red
1384         vref = 0;
1385         turn = 0;
1386     } else if(count <= 15000){
1387         vref = 0;
1388         setEPWM6A_RCServo(Gate_0);
1389     } else if(count <= 18000){
1390         vref = -0.5;
1391     }
1392     else if(count <= 22000){
1393         setEPWM6A_RCServo(Gate_C); //Close the arm
1394         vref=0;
1395         turn=0;
1396     }
1397     break;
1398
1399
1400
1401 default:
1402     break;
1403 }
1404
1405 //Must be called each time into this SWI1 function
1406 PIconrol({uLeft,uRight,vref,turn,LeftWheel,RightWheel});
1407 // These below lines also must be called each time into this SWI1 function
1408 setEPWM1A(uLeft);
1409 setEPWM2A(-uRight);
1410 setF28027EPWM1A(uLeft);
1411 setF28027EPWM2A(-uRight);
1412 LeftWheel_1 = LeftWheel;
1413 RightWheel_1 = RightWheel;
1414
1415 if((timecount%250) == 0) {
1416     DataToLabView.floatData[0] = ROBOTps.x;
1417     DataToLabView.floatData[1] = ROBOTps.y;
1418     DataToLabView.floatData[2] = ROBOTps.theta;

```

```

1419     DataToLabView.floatData[3] = (float)timecount;
1420     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1421     DataToLabView.floatData[5] = (float)RobotState;
1422     DataToLabView.floatData[6] = (float)statePos;
1423     DataToLabView.floatData[7] = LADARfront;
1424     LVsenddata[0] = '$'; // header for LVdata
1425     LVsenddata[1] = '$';
1426     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1427         if (i%2==0) {
1428             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1429         } else {
1430             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1431         }
1432     }
1433     serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1434 }
1435
1436 }
1437 timecount++;
1438 if((timecount%200) == 0)
1439 {
1440     if(doneCal == 0) {
1441         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1442     }
1443     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1444     UARTPrint = 1; // Tell While loop to print
1445 }
1446
1447 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1448 CurrentChip = DAN28027;
1449 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1450 GpioDataRegs.GPCLEAR.bit.GPIO29 = 1;
1451 SpibRegs.SPITXBUF = 0x00DA;
1452 SpibRegs.SPITXBUF = EPwm1A_F28027;
1453 SpibRegs.SPITXBUF = EPwm2A_F28027;
1454
1455 //*****
1456 //
1457 // Restore registers saved:
1458 //
1459 DINT;
1460 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1461
1462 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1463 }
1464
1465 //
1466 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1467 //
1468 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1469 {
1470     // Set interrupt priority:
1471     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1472     IER |= M_INT12;
1473     IER &= MINT12; // Set "global" priority
1474     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1475     PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1476     asm(" NOP");
1477     EINT;
1478
1479 //*****
1480 // Insert SWI ISR Code here.....
1481 // LED1
1482 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1483 if (LADARpingpong == 1) {
1484     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1485     LADARrightfront = 19; // 19 is greater than max feet
1486     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1487         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1488             LADARrightfront = ladar_data[LADARi].distance_ping;
1489         }
1490     }
1491     //DS: LADARleftfront is the min of dist 170, 171, 172, 173, 174
1492     LADARleftfront = 19;
1493     for (LADARi = 166; LADARi <= 170; LADARi++) {
1494         if (ladar_data[LADARi].distance_ping < LADARleftfront) {
1495             LADARleftfront = ladar_data[LADARi].distance_ping;
1496         }
1497     }
1498     // LADARfront is the min of dist 111, 112, 113, 114, 115
1499     LADARfront = 19;
1500     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1501         if (ladar_data[LADARi].distance_ping < LADARfront) {
1502             LADARfront = ladar_data[LADARi].distance_ping;
1503         }
1504     }
1505     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1506     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1507     for (LADARi = 0; LADARi < 228; LADARi++) {
1508
1509         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1510         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1511     }
1512 }
1513 } else if (LADARpingpong == 0) {
1514     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1515     LADARrightfront = 19; // 19 is greater than max feet
1516     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1517         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1518             LADARrightfront = ladar_data[LADARi].distance_pong;
1519         }
1520     }
1521     //DS: LADARleftfront is the min of dist 170, 171, 172, 173, 174
1522     LADARleftfront = 19;
1523     for (LADARi = 166; LADARi <= 170; LADARi++) {
1524         if (ladar_data[LADARi].distance_pong < LADARleftfront) {
1525             LADARleftfront = ladar_data[LADARi].distance_pong;
1526         }
1527     }
1528     // LADARfront is the min of dist 111, 112, 113, 114, 115
1529     LADARfront = 19;
1530     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1531         if (ladar_data[LADARi].distance_pong < LADARfront) {
1532             LADARfront = ladar_data[LADARi].distance_pong;
1533         }
1534     }
1535     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1536     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1537     for (LADARi = 0; LADARi < 228; LADARi++) {
1538
1539         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1540         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1541     }
1542 }
1543 }
1544
1545 //*****
1546 //
1547 // Restore registers saved:
1548 //
1549 DINT;
1550 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1551
1552 }
1553 }
1554
1555 //
1556 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1557 //
1558 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1559 {
1560     // Set interrupt priority:

```

```

1561 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1562 IER |= M_INT12;
1563 IER &= MINT12; // Set "global" priority
1564 PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1565 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1566 _asm(" NOP");
1567 EINT;
1568 RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
1569 // setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1570 // setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1571 // setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
1572 // setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue
1573 // setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate
1574 //#####
1575 // Insert SWI ISR Code here.....
1576
1577 //#####
1578 //
1579 // Restore registers saved:
1580 //
1581 DINT;
1582 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1583
1584 }
1585
1586 // ----- code for CAN start here -----
1587 _interrupt void can_isr(void)
1588 {
1589     int i = 0;
1590
1591     uint32_t status;
1592
1593     //
1594     // Read the CAN interrupt status to find the cause of the interrupt
1595     //
1596     status = CANGetInterruptCause(CANB_BASE);
1597
1598     //
1599     // If the cause is a controller status interrupt, then get the status
1600     //
1601     if(status == CAN_INT_INT0ID_STATUS)
1602     {
1603         //
1604         // Read the controller status. This will return a field of status
1605         // error bits that can indicate various errors. Error processing
1606         // is not done in this example for simplicity. Refer to the
1607         // API documentation for details about the error status bits.
1608         // The act of reading this status will clear the interrupt.
1609         //
1610         status = CANgetStatus(CANB_BASE);
1611     }
1612
1613     //
1614     // Check if the cause is the receive message object 2
1615     //
1616     else if(status == RX_MSG_OBJ_ID_1)
1617     {
1618         //
1619         // Get the received message
1620         //
1621         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1622
1623         for(i = 0; i<2; i++)
1624         {
1625             dis_raw_1[i] = rxMsgData[i];
1626         }
1627
1628         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1629
1630         measure_status_1 = rxMsgData[2];
1631
1632         //
1633         // Getting to this point means that the RX interrupt occurred on
1634         // message object 2, and the message RX is complete. Clear the
1635         // message object interrupt.
1636         //
1637         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1638
1639         //
1640         // Increment a counter to keep track of how many messages have been
1641         // received. In a real application this could be used to set flags to
1642         // indicate when a message is received.
1643         //
1644         rxMsgCount_1++;
1645
1646         //
1647         // Since the message was received, clear any error flags.
1648         //
1649         errorFlag = 0;
1650     }
1651
1652     else if(status == RX_MSG_OBJ_ID_2)
1653     {
1654         //
1655         // Get the received message
1656         //
1657         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1658
1659         for(i = 0; i<2; i++)
1660         {
1661             dis_raw_2[i] = rxMsgData[i];
1662         }
1663
1664         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1665
1666         measure_status_2 = rxMsgData[2];
1667
1668         //
1669         // Getting to this point means that the RX interrupt occurred on
1670         // message object 2, and the message RX is complete. Clear the
1671         // message object interrupt.
1672         //
1673         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1674
1675         //
1676         // Increment a counter to keep track of how many messages have been
1677         // received. In a real application this could be used to set flags to
1678         // indicate when a message is received.
1679         //
1680         rxMsgCount_2++;
1681
1682         //
1683         // Since the message was received, clear any error flags.
1684         //
1685         errorFlag = 0;
1686     }
1687
1688     else if(status == RX_MSG_OBJ_ID_3)
1689     {
1690         //
1691         // Get the received message
1692         //
1693         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1694
1695         for(i = 0; i<4; i++)
1696         {
1697             lightlevel_raw_1[i] = rxMsgData[i];
1698             quality_raw_1[i] = rxMsgData[i+4];
1699         }
1700     }
1701
1702

```

```

1703     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1704     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1705
1706
1707     //
1708     // Getting to this point means that the RX interrupt occurred on
1709     // message object 2, and the message RX is complete. Clear the
1710     // message object interrupt.
1711     //
1712     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1713
1714     //
1715     // Since the message was received, clear any error flags.
1716     //
1717     errorFlag = 0;
1718 }
1719
1720
1721 else if(status == RX_MSG_OBJ_ID_4)
1722 {
1723     //
1724     // Get the received message
1725     //
1726     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1727
1728     for(i = 0; i<4; i++)
1729     {
1730         lightlevel_raw_2[i] = rxMsgData[i];
1731         quality_raw_2[i] = rxMsgData[i+4];
1732     }
1733
1734
1735     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1736     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1737
1738     //
1739     // Getting to this point means that the RX interrupt occurred on
1740     // message object 2, and the message RX is complete. Clear the
1741     // message object interrupt.
1742     //
1743     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1744
1745     //
1746     // Since the message was received, clear any error flags.
1747     //
1748     errorFlag = 0;
1749 }
1750
1751
1752 //
1753 // If something unexpected caused the interrupt, this would handle it.
1754 //
1755 //
1756 else
1757 {
1758     //
1759     // Spurious interrupt handling can go here.
1760     //
1761 }
1762
1763 //
1764 // Clear the global interrupt flag for the CAN interrupt line
1765 //
1766 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1767
1768 //
1769 // Acknowledge this interrupt located in group 9
1770 //
1771 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1772 }
1773 // ----- code for CAN end here----
```

```

1  /* SERIAL.C: This code is designed to act as a low-level serial driver for
2     higher-level programming. Ideally, one could simply call init_serial()
3     to initialize the serial port, then use serial_send("data", 4) to send
4     an array of data (8-bit unsigned character strings).
5
6     WRITTEN BY : Paul Miller <pamiller@uiuc.edu>
7     $Id: serial.c,v 1.4 2003/08/08 16:08:56 paul Exp $
8  */
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <stdarg.h>
12 #include <string.h>
13 #include <math.h>
14 #include <limits.h>
15
16 #include "F28x_Project.h"      // Device Headerfile and Examples Include File
17 #include <buffer.h>
18 #include <F28379dSerial.h>
19 #include <F2837xD_sci.h>
20 #include "SE423Lib.h"
21
22 serialSCIA_t SerialA;
23 serialSCIB_t SerialB;
24 serialSCIC_t SerialC;
25 serialSCID_t SerialD;
26
27 char RXAdata = 0;
28 char RXBdata = 0;
29 char RxCdata = 0;
30 char RxDdata = 0;
31 uint32_t numRXA = 0;
32 uint32_t numRXB = 0;
33 uint32_t numRXC = 0;
34 uint32_t numRXD = 0;
35
36 char buf[477]; //store all the data
37 uint16_t dis_index = 0;
38 uint16_t G_count = 0; //number of thing received after 'G'
39 uint16_t LF_count = 0; //number of consecutive line feed
40 uint16_t buf_index = 0; //for debug
41 uint16_t get_dis_first; //if it is the first part of the two parts data
42 uint16_t print_dis_index=0;
43 uint32_t numRCXerrors = 0;
44 int16_t tempscid = 0;
45 int16_t distance = 0;
46 float checkdist = 0;
47
48 uint16_t COMA_state = 0;
49 uint16_t COMALSB = 0;
50
51 uint16_t received_CAM_countThreshold1 = 0;
52 uint16_t NewCAMDataThreshold1 = 0; // Flag new data
53 CAMRecFloats_t DataFromCameraThreshold1;
54 float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
55
56 uint16_t received_CAM_countThreshold2 = 0;
57 uint16_t NewCAMDataThreshold2 = 0; // Flag new data
58 CAMRecFloats_t DataFromCameraThreshold2;
59 float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
60
61 //EC: Declaration added for the third color
62 uint16_t received_CAM_countThreshold3 = 0;
63 uint16_t NewCAMDataThreshold3 = 0; // Flag new data
64 CAMRecFloats_t DataFromCameraThreshold3;
65 float fromCAMvaluesThreshold3[CAMNUM_FROM_FLOATS];
66
67 uint16_t received_CAM_countAprilTag1 = 0;
68 uint16_t NewCAMDataAprilTag1 = 0; // Flag new data
69 CAMRecFloats_t DataFromCameraAprilTag1;
70 float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
71
72 uint16_t tempLSB = 0;
73 LVRecFloats_t DataFromLabView;
74 LVSendFloats_t DataToLabView;
75 uint16_t received_LV_count = 0;
76 uint16_t NewLVData = 0;
77 char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
78 float fromLVvalues[LVNUM_TOFROM_FLOATS];
79
80 CMDRecFloats_t DataFromLinuxCMD;
81 uint16_t received_CMD_count = 0;
82 uint16_t newLinuxCommands = 0;
83 float LinuxCommands[CMDNUM_FROM_FLOATS];
84 uint16_t com_state = 0; //the state of receive from pi (SCID)
85 uint16_t LADARpingpong = 0;
86 uint16_t read_ladar_state = 0; //states for ladar reading, initialized as wait for S command
87 datapts_ladar_data[228]; //distance data from ladar
88

```

```

89 uint16_t new_optitrack = 0;
90 float Optitrackdata[OPTITRACKDATASIZE]; // x, y, heading, trackableID, framecount
91 uint16_t received_opti_count = 0; // the count for receiving optitrack data
92 optiData_t optirx;
93
94 char path_received[81];
95 int16_t received_path_count = 0;
96 int16_t newAstarPath = 0;
97 float ladar161distance = 0;
98 float ladar199distance = 0;
99
100
101 // for SerialA
102 uint16_t init_serialSCIA(serialSCIA_t *s, uint32_t baud)
103 {
104     volatile struct SCI_REGS *sci;
105     uint32_t clk;
106
107     if (s == &SerialA) {
108         sci = &SciaRegs;
109         s->sci = sci;
110
111         init_bufferSCIA(&s->TX);
112
113         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 0);
114         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
115         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 0);
116         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
117
118     } else {
119         return 1;
120     }
121
122     /* init for standard baud, 8N1 comm */
123     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
124     sci->SCICCR.all = 0x0;
125     sci->SCICTL1.all = 0x0;
126     sci->SCICTL2.all = 0x0;
127     sci->SCIPRI.all = 0x0;
128     clk = LSPCLK_HZ; // set baud rate
129     clk /= baud*8;
130     clk--;
131     sci->SCILBAUD.all = clk & 0xFF;
132     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
133
134     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
135     sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
136     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
137     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
138     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
139
140     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
141
142     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
143     sci->SCIFFTX.bit.TXFIFORESET = 0;
144     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
145     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
146     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
147     sci->SCIFFTX.bit.TXFIFORESET = 1;
148
149     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
150     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
151     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
152     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
153     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
154
155     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
156     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
157
158     sci->SCICTL1.bit.TXWAKE = 0;
159     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
160     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
161     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
162     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
163     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
164
165     /* enable PIE interrupts */
166     if (s == &SerialA) {
167         PieCtrlRegs.PIEIER9.bit.INTx1 = 1;
168         PieCtrlRegs.PIEIER9.bit.INTx2 = 1;
169         IER |= (M_INT9);
170         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
171         GPIO_SetupPinMux(43, GPIO_MUX_CPU1, 15);
172         GPIO_SetupPinOptions(43, GPIO_INPUT, GPIO_PULLUP);
173         GPIO_SetupPinMux(42, GPIO_MUX_CPU1, 15);
174         GPIO_SetupPinOptions(42, GPIO_OUTPUT, GPIO_PUSHPULL);
175
176     }
177
178     return 0;
179 }

```



```

180
181 void uninit_serialSCIA(serialSCIA_t *s)
182 {
183     volatile struct SCI_REGS *sci = s->sci;
184
185     /* disable PIE interrupts */
186     if (s == &SerialA) {
187         PieCtrlRegs.PIEIER9.bit.INTx1 = 0;
188         PieCtrlRegs.PIEIER9.bit.INTx2 = 0;
189         IER &= ~M_INT9;
190     }
191     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
192     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
193     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
194
195     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
196     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
197 }
198
199
200
201 /*****
202  * SERIAL_SEND()
203  *
204  * "User level" function to send data via serial. Return value is the
205  * length of data successfully copied to the TX buffer.
206  *****/
207 uint16_t serial_sendSCIA(serialSCIA_t *s, char *data, uint16_t len)
208 {
209     uint16_t i = 0;
210     if (len && s->TX.size < BUF_SIZESCIA) {
211         for (i = 0; i < len; i++) {
212             if (buf_writeSCIA_1(&s->TX, data[i] & 0x00FF) != 0) break;
213         }
214         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
215         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
216     }
217     return i;
218 }
219
220 // For SerialB
221 uint16_t init_serialSCIB(serialSCIB_t *s, uint32_t baud)
222 {
223     volatile struct SCI_REGS *sci;
224     uint32_t clk;
225
226     if (s == &SerialB) {
227         sci = &ScibRegs;
228         s->sci = sci;
229
230         init_bufferSCIB(&s->TX);
231
232         GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 0);
233         GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
234         GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 0);
235         GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
236
237     } else {
238         return 1;
239     }
240 }
241
242 /* init for standard baud, 8N1 comm */
243 sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
244 sci->SCICCR.all = 0x0;
245 sci->SCICTL1.all = 0x0;
246 sci->SCICTL2.all = 0x0;
247 sci->SCIPRI.all = 0x0;
248 clk = LSPCLK_HZ; // set baud rate
249 clk /= baud*8;
250 clk--;
251 sci->SCILBAUD.all = clk & 0xFF;
252 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
253
254 sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
255 sci->SCICCR.bit.PARIYENA = 0; // (N) disable parity calculation
256 sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
257 sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
258 sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
259
260 sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
261
262 sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
263 sci->SCIFFTX.bit.TXFIFORESET = 0;
264 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
265 sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
266 sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
267 sci->SCIFFTX.bit.TXFIFORESET = 1;
268
269 sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
270 sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag

```

```

271 sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
272 sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
273 sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
274
275 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
276 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
277
278 sci->SCICTL1.bit.TXWAKE = 0;
279 sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
280 sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
281 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
282 sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
283 sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
284
285 /* enable PIE interrupts */
286 if (s == &SerialB) {
287     PieCtrlRegs.PIEIER9.bit.INTx3 = 1;
288     PieCtrlRegs.PIEIER9.bit.INTx4 = 1;
289     IER |= (M_INT9);
290     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP9);
291     GPIO_SetupPinMux(15, GPIO_MUX_CPU1, 2);
292     GPIO_SetupPinOptions(15, GPIO_INPUT, GPIO_PULLUP);
293     GPIO_SetupPinMux(14, GPIO_MUX_CPU1, 2);
294     GPIO_SetupPinOptions(14, GPIO_OUTPUT, GPIO_PUSHPULL);
295 }
296
297
298 return 0;
299 }
300
301 void uninit_serialSCIB(serialSCIB_t *s)
302 {
303     volatile struct SCI_REGS *sci = s->sci;
304
305     /* disable PIE interrupts */
306     if (s == &SerialB) {
307         PieCtrlRegs.PIEIER9.bit.INTx3 = 0;
308         PieCtrlRegs.PIEIER9.bit.INTx4 = 0;
309         IER &= ~M_INT9;
310     }
311
312     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
313     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
314     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
315
316     sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
317     sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
318 }
319
320
321
322 /*****
323  * SERIAL_SEND()
324  *
325  * "User level" function to send data via serial. Return value is the
326  * length of data successfully copied to the TX buffer.
327  *****/
328 uint16_t serial_sendSCIB(serialSCIB_t *s, char *data, uint16_t len)
329 {
330     uint16_t i = 0;
331     if (len && s->TX.size < BUF_SIZE_SCIB) {
332         for (i = 0; i < len; i++) {
333             if (buf_writeSCIB_1(&s->TX, data[i] & 0x00FF) != 0) break;
334         }
335         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
336         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
337     }
338     return i;
339 }
340
341 // for SerialC
342 uint16_t init_serialSCIC(serialSCIC_t *s, uint32_t baud)
343 {
344     volatile struct SCI_REGS *sci;
345     uint32_t clk;
346
347     if (s == &SerialC) {
348         sci = &ScicRegs;
349         s->sci = sci;
350
351         init_bufferSCIC(&s->TX);
352         GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 0);
353         GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
354         GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 0);
355         GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
356
357     } else {
358         return 1;
359     }
360
361     /* init for standard baud, 8N1 comm */

```

```

362 sci->SCICTL1.bit.SWRESET = 0;          // init SCI state machines and opt flags
363 sci->SCICCR.all = 0x0;
364 sci->SCICTL1.all = 0x0;
365 sci->SCICTL2.all = 0x0;
366 sci->SCIPRI.all = 0x0;
367 clk = LSPCLK_HZ;                      // set baud rate
368 clk /= baud*8;
369 clk--;
370 sci->SCILBAUD.all = clk & 0xFF;
371 sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
372
373 sci->SCICCR.bit.SCICHAR = 0x7;          // (8) 8 bits per character
374 sci->SCICCR.bit.PARITYENA = 0;          // (N) disable parity calculation
375 sci->SCICCR.bit.STOPBITS = 0;           // (1) transmit 1 stop bit
376 sci->SCICCR.bit.LOOPBKENA = 0;          // disable loopback test
377 sci->SCICCR.bit.ADDRIDLE_MODE = 0;      // idle-line mode (non-multiprocessor SCI comm)
378
379 sci->SCIFFCT.bit.FFTXDLY = 0;           // TX: zero-delay
380
381 sci->SCIFFTX.bit.SCIFFENA = 1;           // enable SCI fifo enhancements
382 sci->SCIFFTX.bit.TXFIFORESET = 0;
383 sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels  ??? is this correct
384 sci->SCIFFTX.bit.TXFFINTCLR = 1;        // TX: clear interrupt flag
385 sci->SCIFFTX.bit.TXFFIENA = 0;          // TX: disable fifo interrupt
386 sci->SCIFFTX.bit.TXFIFORESET = 1;
387
388 sci->SCIFFRX.bit.RXFIFORESET = 0;        // RX: fifo reset
389 sci->SCIFFRX.bit.RXFFINTCLR = 1;        // RX: clear interrupt flag
390 sci->SCIFFRX.bit.RXFFIENA = 1;          // RX: enable fifo interrupt
391 sci->SCIFFRX.bit.RXFFIL = 0x1;          // RX: fifo interrupt
392 sci->SCIFFRX.bit.RXFIFORESET = 1;        // RX: re-enable fifo
393
394 sci->SCICTL2.bit.RXBKINTENA = 0;         // disable receiver/error interrupt
395 sci->SCICTL2.bit.TXINTENA = 0;          // disable transmitter interrupt
396
397 sci->SCICTL1.bit.TXWAKE = 0;
398 sci->SCICTL1.bit.SLEEP = 0;             // disable sleep mode
399 sci->SCICTL1.bit.RXENA = 1;             // enable SCI receiver
400 sci->SCICTL1.bit.RXERRINTENA = 0;       // disable receive error interrupt
401 sci->SCICTL1.bit.TXENA = 1;             // enable SCI transmitter
402 sci->SCICTL1.bit.SWRESET = 1;           // re-enable SCI
403
404 /* enable PIE interrupts */
405 if (s == &SerialC) {
406     PieCtrlRegs.PIEIER8.bit.INTx5 = 1;
407     PieCtrlRegs.PIEIER8.bit.INTx6 = 1;
408     PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
409     IER |= (M_INT8);
410     GPIO_SetupPinMux(139, GPIO_MUX_CPU1, 6);
411     GPIO_SetupPinOptions(139, GPIO_INPUT, GPIO_PULLUP);
412     GPIO_SetupPinMux(56, GPIO_MUX_CPU1, 6);
413     GPIO_SetupPinOptions(56, GPIO_OUTPUT, GPIO_PUSHPULL);
414 }
415
416 return 0;
417 }
418
419 void uninit_serialSCIC(serialSCIC_t *s)
420 {
421     volatile struct SCI_REGS *sci = s->sci;
422
423     /* disable PIE interrupts */
424     if (s == &SerialC) {
425         PieCtrlRegs.PIEIER8.bit.INTx5 = 0;
426         PieCtrlRegs.PIEIER8.bit.INTx6 = 0;
427         IER &= ~M_INT8;
428     }
429
430     sci->SCICTL1.bit.RXERRINTENA = 0;     // disable receive error interrupt
431     sci->SCICTL2.bit.RXBKINTENA = 0;     // disable receiver/error interrupt
432     sci->SCICTL2.bit.TXINTENA = 0;        // disable transmitter interrupt
433
434     sci->SCICTL1.bit.RXENA = 0;           // disable SCI receiver
435     sci->SCICTL1.bit.TXENA = 0;           // disable SCI transmitter
436 }
437
438
439
440 /*****
441  * SERIAL_SEND()
442  *
443  * "User level" function to send data via serial. Return value is the
444  * length of data successfully copied to the TX buffer.
445  *****/
446 uint16_t serial_sendSCIC(serialSCIC_t *s, char *data, uint16_t len)
447 {
448     uint16_t i = 0;
449     if (len && s->TX.size < BUF_SIZE_SCIC) {
450         for (i = 0; i < len; i++) {
451             if (buf_writeSCIC_1(&s->TX, data[i] & 0x00FF) != 0) break;
452         }
453     }

```

```

453     s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
454     s->sci->SCIFFTX.bit.TXFFIENA = 1;    // TX: enable fifo interrupt
455 }
456 return i;
457 }
458
459 // For Serial D
460 uint16_t init_serialSCID(serialSCID_t *s, uint32_t baud)
461 {
462     volatile struct SCI_REGS *sci;
463     uint32_t clk;
464
465     if (s == &SerialD) {
466         sci = &ScidRegs;
467         s->sci = sci;
468
469         init_bufferSCID(&s->TX);
470         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 0);
471         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
472         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 0);
473         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
474     }
475     else {
476         return 1;
477     }
478
479     /* init for standard baud,8N1 comm */
480     sci->SCICTL1.bit.SWRESET = 0; // init SCI state machines and opt flags
481     sci->SCICCR.all = 0x0;
482     sci->SCICTL1.all = 0x0;
483     sci->SCICTL2.all = 0x0;
484     sci->SCIPRI.all = 0x0;
485     clk = LSPCLK_HZ; // set baud rate
486     clk /= baud*8;
487     clk--;
488     sci->SCILBAUD.all = clk & 0xFF;
489     sci->SCIHBAUD.all = (clk >> 8) & 0xFF;
490
491     sci->SCICCR.bit.SCICCHAR = 0x7; // (8) 8 bits per character
492     sci->SCICCR.bit.PARITYENA = 0; // (N) disable parity calculation
493     sci->SCICCR.bit.STOPBITS = 0; // (1) transmit 1 stop bit
494     sci->SCICCR.bit.LOOPBKENA = 0; // disable loopback test
495     sci->SCICCR.bit.ADDRIDLE_MODE = 0; // idle-line mode (non-multiprocessor SCI comm)
496
497     sci->SCIFFCT.bit.FFTXDLY = 0; // TX: zero-delay
498
499     sci->SCIFFTX.bit.SCIFFENA = 1; // enable SCI fifo enhancements
500     sci->SCIFFTX.bit.TXFIFORESET = 0;
501     sci->SCIFFTX.bit.TXFFIL = 0x0; // TX: fifo interrupt at all levels ??? is this correct
502     sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
503     sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
504     sci->SCIFFTX.bit.TXFIFORESET = 1;
505
506     sci->SCIFFRX.bit.RXFIFORESET = 0; // RX: fifo reset
507     sci->SCIFFRX.bit.RXFFINTCLR = 1; // RX: clear interrupt flag
508     sci->SCIFFRX.bit.RXFFIENA = 1; // RX: enable fifo interrupt
509     sci->SCIFFRX.bit.RXFFIL = 0x1; // RX: fifo interrupt
510     sci->SCIFFRX.bit.RXFIFORESET = 1; // RX: re-enable fifo
511
512     sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
513     sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
514
515     sci->SCICTL1.bit.TXWAKE = 0;
516     sci->SCICTL1.bit.SLEEP = 0; // disable sleep mode
517     sci->SCICTL1.bit.RXENA = 1; // enable SCI receiver
518     sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
519     sci->SCICTL1.bit.TXENA = 1; // enable SCI transmitter
520     sci->SCICTL1.bit.SWRESET = 1; // re-enable SCI
521
522     /* enable PIE interrupts */
523     if (s == &SerialD) {
524         PieCtrlRegs.PIEIER8.bit.INTx7 = 1;
525         PieCtrlRegs.PIEIER8.bit.INTx8 = 1;
526         PieCtrlRegs.PIEACK.all = (PIEACK_GROUP8);
527         IER |= (M_INT8);
528         GPIO_SetupPinMux(105, GPIO_MUX_CPU1, 6);
529         GPIO_SetupPinOptions(105, GPIO_INPUT, GPIO_PULLUP);
530         GPIO_SetupPinMux(104, GPIO_MUX_CPU1, 6);
531         GPIO_SetupPinOptions(104, GPIO_OUTPUT, GPIO_PUSHPULL);
532     }
533 }
534
535 return 0;
536 }
537
538 void uninit_serialSCID(serialSCID_t *s)
539 {
540     volatile struct SCI_REGS *sci = s->sci;
541
542     /* disable PIE interrupts */
543     if (s == &SerialD) {

```

```

544     PieCtrlRegs.PIEIER8.bit.INTx7 = 0;
545     PieCtrlRegs.PIEIER8.bit.INTx8 = 0;
546     IER &= ~M_INT8;
547 }
548
549 sci->SCICTL1.bit.RXERRINTENA = 0; // disable receive error interrupt
550 sci->SCICTL2.bit.RXBKINTENA = 0; // disable receiver/error interrupt
551 sci->SCICTL2.bit.TXINTENA = 0; // disable transmitter interrupt
552
553 sci->SCICTL1.bit.RXENA = 0; // disable SCI receiver
554 sci->SCICTL1.bit.TXENA = 0; // disable SCI transmitter
555 }
556
557
558
559 /*****
560 * SERIAL_SEND()
561 *
562 * "User level" function to send data via serial. Return value is the
563 * length of data successfully copied to the TX buffer.
564 *****/
565 uint16_t serial_sendSCID(serialSCID_t *s, char *data, uint16_t len)
566 {
567     uint16_t i = 0;
568     if (len && s->TX.size < BUF_SIZE_SCID) {
569         for (i = 0; i < len; i++) {
570             if (buf_writeSCID_1(&s->TX, data[i] & 0x00FF) != 0) break;
571         }
572         s->sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
573         s->sci->SCIFFTX.bit.TXFFIENA = 1; // TX: enable fifo interrupt
574     }
575     return i;
576 }
577
578
579
580 /*****
581 * TXxINT_DATA_SENT()
582 *
583 * Executed when transmission is ready for additional data. These functions
584 * read the next char of data and put it in the TXBUF register for transfer.
585 *****/
586 #ifdef _FLASH
587 #pragma CODE_SECTION(TXAINT_data_sent, ".TI.ramfunc");
588 #endif
589 __interrupt void TXAINT_data_sent(void)
590 {
591     char data;
592     if (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0) {
593         while ( (buf_readSCIA_1(&SerialA.TX, 0, &data) == 0)
594             && (SerialA.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
595             buf_removeSCIA(&SerialA.TX, 1);
596             SerialA.sci->SCITXBUF.all = data;
597         }
598     } else {
599         SerialA.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
600     }
601     SerialA.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
602     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
603 }
604
605
606 //for serialB
607 #ifdef _FLASH
608 #pragma CODE_SECTION(TXBINT_data_sent, ".TI.ramfunc");
609 #endif
610 __interrupt void TXBINT_data_sent(void)
611 {
612     char data;
613     if (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0) {
614         while ( (buf_readSCIB_1(&SerialB.TX, 0, &data) == 0)
615             && (SerialB.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
616             buf_removeSCIB(&SerialB.TX, 1);
617             SerialB.sci->SCITXBUF.all = data;
618         }
619     } else {
620         SerialB.sci->SCIFFTX.bit.TXFFIENA = 0; // TX: disable fifo interrupt
621     }
622     SerialB.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
623     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
624 }
625
626
627 //for serialC
628 #ifdef _FLASH
629 #pragma CODE_SECTION(TXCINT_data_sent, ".TI.ramfunc");
630 #endif
631 __interrupt void TXCINT_data_sent(void)
632 {
633     char data;
634     if (buf_readSCIC_1(&SerialC.TX, 0, &data) == 0) {

```

```

635     while ( (buf_readSCIC_1(&SerialC.TX,0,&data) == 0)
636             && (SerialC.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
637         buf_removeSCIC(&SerialC.TX, 1);
638         SerialC.sci->SCITXBUF.all = data;
639     }
640 } else {
641     SerialC.sci->SCIFFTX.bit.TXFFIENA = 0;    // TX: disable fifo interrupt
642 }
643 SerialC.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
644 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
645 }
646
647 //for serialD
648 #ifdef _FLASH
649 #pragma CODE_SECTION(TXDINT_data_sent, ".TI.ramfunc");
650 #endif
651 __interrupt void TXDINT_data_sent(void)
652 {
653     char data;
654     if (buf_readSCID_1(&SerialD.TX,0,&data) == 0) {
655         while ( (buf_readSCID_1(&SerialD.TX,0,&data) == 0)
656                 && (SerialD.sci->SCIFFTX.bit.TXFFST != 0x10) ) {
657             buf_removeSCID(&SerialD.TX, 1);
658             SerialD.sci->SCITXBUF.all = data;
659         }
660     } else {
661         SerialD.sci->SCIFFTX.bit.TXFFIENA = 0;    // TX: disable fifo interrupt
662     }
663     SerialD.sci->SCIFFTX.bit.TXFFINTCLR = 1; // TX: clear interrupt flag
664     PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
665 }
666
667 //for SerialA
668 #ifdef _FLASH
669 #pragma CODE_SECTION(RXAINT_recv_ready, ".TI.ramfunc");
670 #endif
671 __interrupt void RXAINT_recv_ready(void)
672 {
673     RXAdata = SciaRegs.SCIRXBUF.all;
674
675     /* SCI PE or FE error */
676     if (RXAdata & 0xC000) {
677         SciaRegs.SCICTL1.bit.SWRESET = 0;
678         SciaRegs.SCICTL1.bit.SWRESET = 1;
679         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
680         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;
681     } else {
682         RXAdata = RXAdata & 0x00FF;
683         if (COMA_state == 0) {
684
685             if (RXAdata == '*') {
686                 COMA_state = 1; //for receiving path
687             } else {
688                 COMA_state = 0;
689             }
690         } else if (COMA_state == 1) {
691
692             if (RXAdata == '*') {
693                 COMA_state = 10; // into the read mode
694                 received_CAM_countThreshold1 = 0;
695             } else if (RXAdata == '!') {
696                 COMA_state = 20; // into the read mode
697                 received_CAM_countThreshold2 = 0;
698             } else if (RXAdata == '$') {
699                 COMA_state = 30; // into the read mode
700                 received_CAM_countAprilTag1 = 0;
701             } else if (RXAdata == '&') {
702                 COMA_state = 40; //EC: into the read mode
703                 received_CAM_countThreshold3 = 0;
704             }
705             else {
706                 COMA_state = 0;
707             }
708         } else if (COMA_state == 10) {
709
710             if ((received_CAM_countThreshold1 % 2) == 0) {
711                 COMALSB = RXAdata;
712             } else {
713                 DataFromCameraThreshold1.rawData[received_CAM_countThreshold1/2] = (RXAdata << 8) | COMALSB;
714             }
715             received_CAM_countThreshold1++;
716             if (received_CAM_countThreshold1 >= 4*CAMNUM_FROM_FLOATS) {
717                 received_CAM_countThreshold1 = 0;
718                 COMA_state = 0;
719                 fromCAMvaluesThreshold1[0] = DataFromCameraThreshold1.floatData[0];
720                 fromCAMvaluesThreshold1[1] = DataFromCameraThreshold1.floatData[1];
721                 fromCAMvaluesThreshold1[2] = DataFromCameraThreshold1.floatData[2];
722                 fromCAMvaluesThreshold1[3] = DataFromCameraThreshold1.floatData[3];
723                 fromCAMvaluesThreshold1[4] = DataFromCameraThreshold1.floatData[4];
724                 fromCAMvaluesThreshold1[5] = DataFromCameraThreshold1.floatData[5];
725                 fromCAMvaluesThreshold1[6] = DataFromCameraThreshold1.floatData[6];

```



```

726         fromCAMvaluesThreshold1[7] = DataFromCameraThreshold1.floatData[7];
727         fromCAMvaluesThreshold1[8] = DataFromCameraThreshold1.floatData[8];
728         NewCAMDataThreshold1 = 1; // Flag new data
729     }
730 } else if (COMA_state == 20) {
731     if ((received_CAM_countThreshold2 % 2) == 0) {
732         COMALSB = RXAdata;
733     } else {
734         DataFromCameraThreshold2.rawData[received_CAM_countThreshold2/2] = (RXAdata << 8) | COMALSB;
735     }
736     received_CAM_countThreshold2++;
737     if (received_CAM_countThreshold2 >= 4*CAMNUM_FROM_FLOATS) {
738         received_CAM_countThreshold2 = 0;
739         COMA_state = 0;
740         fromCAMvaluesThreshold2[0] = DataFromCameraThreshold2.floatData[0];
741         fromCAMvaluesThreshold2[1] = DataFromCameraThreshold2.floatData[1];
742         fromCAMvaluesThreshold2[2] = DataFromCameraThreshold2.floatData[2];
743         fromCAMvaluesThreshold2[3] = DataFromCameraThreshold2.floatData[3];
744         fromCAMvaluesThreshold2[4] = DataFromCameraThreshold2.floatData[4];
745         fromCAMvaluesThreshold2[5] = DataFromCameraThreshold2.floatData[5];
746         fromCAMvaluesThreshold2[6] = DataFromCameraThreshold2.floatData[6];
747         fromCAMvaluesThreshold2[7] = DataFromCameraThreshold2.floatData[7];
748         fromCAMvaluesThreshold2[8] = DataFromCameraThreshold2.floatData[8];
749         NewCAMDataThreshold2 = 1; // Flag new data
750     }
751 } else if (COMA_state == 30) {
752     if ((received_CAM_countAprilTag1 % 2) == 0) {
753         COMALSB = RXAdata;
754     } else {
755         DataFromCameraAprilTag1.rawData[received_CAM_countAprilTag1/2] = (RXAdata << 8) | COMALSB;
756     }
757     received_CAM_countAprilTag1++;
758     if (received_CAM_countAprilTag1 >= 4*CAMNUM_FROM_FLOATS) {
759         received_CAM_countAprilTag1 = 0;
760         COMA_state = 0;
761         fromCAMvaluesAprilTag1[0] = DataFromCameraAprilTag1.floatData[0];
762         fromCAMvaluesAprilTag1[1] = DataFromCameraAprilTag1.floatData[1];
763         fromCAMvaluesAprilTag1[2] = DataFromCameraAprilTag1.floatData[2];
764         fromCAMvaluesAprilTag1[3] = DataFromCameraAprilTag1.floatData[3];
765         fromCAMvaluesAprilTag1[4] = DataFromCameraAprilTag1.floatData[4];
766         fromCAMvaluesAprilTag1[5] = DataFromCameraAprilTag1.floatData[5];
767         fromCAMvaluesAprilTag1[6] = DataFromCameraAprilTag1.floatData[6];
768         fromCAMvaluesAprilTag1[7] = DataFromCameraAprilTag1.floatData[7];
769         fromCAMvaluesAprilTag1[8] = DataFromCameraAprilTag1.floatData[8];
770         NewCAMDataAprilTag1 = 1; // Flag new data
771     }
772 } else if (COMA_state == 40) {
773     if ((received_CAM_countThreshold3 % 2) == 0) {
774         COMALSB = RXAdata;
775     } else {
776         DataFromCameraThreshold3.rawData[received_CAM_countThreshold3/2] = (RXAdata << 8) | COMALSB;
777     }
778     received_CAM_countThreshold3++;
779     if (received_CAM_countThreshold3 >= 4*CAMNUM_FROM_FLOATS) {
780         received_CAM_countThreshold3 = 0;
781         COMA_state = 0;
782         fromCAMvaluesThreshold3[0] = DataFromCameraThreshold3.floatData[0];
783         fromCAMvaluesThreshold3[1] = DataFromCameraThreshold3.floatData[1];
784         fromCAMvaluesThreshold3[2] = DataFromCameraThreshold3.floatData[2];
785         fromCAMvaluesThreshold3[3] = DataFromCameraThreshold3.floatData[3];
786         fromCAMvaluesThreshold3[4] = DataFromCameraThreshold3.floatData[4];
787         fromCAMvaluesThreshold3[5] = DataFromCameraThreshold3.floatData[5];
788         fromCAMvaluesThreshold3[6] = DataFromCameraThreshold3.floatData[6];
789         fromCAMvaluesThreshold3[7] = DataFromCameraThreshold3.floatData[7];
790         fromCAMvaluesThreshold3[8] = DataFromCameraThreshold3.floatData[8];
791         NewCAMDataThreshold3 = 1; // Flag new data
792     }
793 }
794
795     numRXA ++;
796 }
797
798 SciaRegs.SCIFFRX.bit.RXFFINTCLR = 1;
799 PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
800
801 }
802
803 //for SerialB
804 #ifdef _FLASH
805 #pragma CODE_SECTION(RXBINT_recv_ready, ".TI.ramfunc");
806 #endif
807 __interrupt void RXBINT_recv_ready(void)
808 {
809     RXBdata = ScibRegs.SCIRXBUF.all;
810
811     /* SCI PE or FE error */
812     if (RXBdata & 0xC000) {
813         ScibRegs.SCICTL1.bit.SWRESET = 0;
814         ScibRegs.SCICTL1.bit.SWRESET = 1;
815         SciaRegs.SCIFFRX.bit.RXFIFORESET = 0;
816         SciaRegs.SCIFFRX.bit.RXFIFORESET = 1;

```



```

817     } else {
818         RXBdata = RXBdata & 0x00FF;
819         // Do something with recieved character
820         numRXB ++;
821     }
822
823     ScibRegs.SCIFFRX.bit.RXFFINTCLR = 1;
824     PieCtrlRegs.PIEACK.all = PIEACK_GROUP9;
825 }
826
827
828 // for SerialC
829 #ifdef _FLASH
830 #pragma CODE_SECTION(RXCINT_recv_ready, ".TI.ramfunc");
831 #endif
832 __interrupt void RXCINT_recv_ready(void)
833 {
834     RXCdata = ScicRegs.SCIRXBUF.all;
835
836     /* SCI PE or FE error */
837     if (RXCdata & 0xC000) {
838         ScicRegs.SCICTL1.bit.SWRESET = 0;
839         ScicRegs.SCICTL1.bit.SWRESET = 1;
840         ScicRegs.SCIFFRX.bit.RXFIFORESET = 0;
841         ScicRegs.SCIFFRX.bit.RXFIFORESET = 1;
842     } else {
843         RXCdata = RXCdata & 0x00FF;
844         if (read_ladar_state == 0) { //wait for the starting char 'G'
845             buf_index = 0;
846             if (RXCdata == 'G') {
847                 read_ladar_state = 1;
848                 G_count = 1;
849             }
850         } else if (read_ladar_state == 1) { //get the headers
851             G_count += 1; //count the number of headers
852             //check the status code and if it is not '0', wait for the next packet
853             if (G_count == 11) {
854                 if (RXCdata != '0') {
855                     read_ladar_state = 0;
856                 }
857             }
858             if (G_count >= 12) { //there are total 12 things before the data
859                 read_ladar_state = 2;
860                 get_dis_first = 1;
861                 dis_index = 0;
862             }
863         } else if (read_ladar_state == 2) { //get actual data
864             if (RXCdata == 0x0a) { //check for line feed
865                 LF_count += 1;
866             } else {
867                 LF_count = 0;
868                 if (get_dis_first == 1) {
869                     tempscid = (RXCdata - 0x30) << 6; //first data is the top 6 bit of the distance reading
870                     get_dis_first = 0;
871                 } else if (get_dis_first == 0) {
872                     distance = tempscid + (RXCdata - 0x30); //second data is the bottom 6 bit of the distance reading
873                     if (distance == 0 || distance == 6) { //error code
874                         distance = LADAR_MAX_READING; //set to the max reading
875                     } else if (distance == 16) { //distance is 4096mm
876                         distance = 4096;
877                     }
878                     if (LADARpingpong == 0) {
879                         //ladar_data[dis_index%228].distance_ping = distance*0.00328084; // fill in the distance_ping array
880                         checkdist = distance*0.00328084;
881                         if ((checkdist > 0.25) && (checkdist < 20.0)) {
882                             ladar_data[dis_index].distance_ping = checkdist; // fill in the distance_ping array
883                             if (dis_index == 161) {
884                                 ladar161distance = checkdist;
885                             }
886                             if (dis_index == 199) {
887                                 ladar199distance = checkdist;
888                             }
889                         } else {
890                             numRXErrors++;
891                             ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_pong;
892                             if (dis_index == 161) {
893                                 ladar161distance = ladar_data[dis_index].distance_pong;
894                             }
895                             if (dis_index == 199) {
896                                 ladar199distance = ladar_data[dis_index].distance_pong;
897                             }
898                         }
899                     }
900                 } else if (LADARpingpong == 1) {
901                     //ladar_data[dis_index%228].distance_pong = distance*0.00328084; // fill in the distance_pong array
902                     checkdist = distance*0.00328084;
903                     if ((checkdist >= 0.25) && (checkdist < 20.0)) {
904                         ladar_data[dis_index].distance_pong = checkdist; // fill in the distance_ping array
905                         if (dis_index == 161) {
906                             ladar161distance = checkdist;
907                         }
908                     }
909                 }
910             }
911         }
912     }
913 }

```

```

908         if (dis_index == 199) {
909             ladar199distance = checkdist;
910         }
911     } else {
912         numRCXerrors++;
913         ladar_data[dis_index].distance_ping = ladar_data[dis_index].distance_ping;
914         if (dis_index == 161) {
915             ladar161distance = ladar_data[dis_index].distance_ping;
916         }
917         if (dis_index == 199) {
918             ladar199distance = ladar_data[dis_index].distance_ping;
919         }
920     }
921 }
922 dis_index += 1;
923 get_dis_first = 1;
924 }
925 }
926 if (LF_count == 2) { //two consecutive line feed means the end of the current data stream
927     read_ladar_state = 0;
928     print_dis_index = dis_index;
929     PostSWI2(); // Manually cause the interrupt for the SWI2
930     LADARpingpong = 1-LADARpingpong;
931 }
932 }
933 buf[buf_index % 477] = RXCdata;//fill in the buf array, debug purpose
934 buf_index++;
935 numRXC ++;
936 }
937 }
938
939 ScicRegs.SCIFFRX.bit.RXFFINTCLR = 1;
940 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
941 }
942
943
944 //for SerialD
945 #ifdef _FLASH
946 #pragma CODE_SECTION(RXDINT_recv_ready, ".TI.ramfunc");
947 #endif
948 __interrupt void RXDINT_recv_ready(void)
949 {
950     uint16_t i = 0;//for loop
951     RXDdata = ScidRegs.SCIRXBUF.all;
952
953     /* SCI PE or FE error */
954     if (RXDdata & 0xC000) {
955         ScidRegs.SCICTL1.bit.SWRESET = 0;
956         ScidRegs.SCICTL1.bit.SWRESET = 1;
957         ScidRegs.SCIFFRX.bit.RXFIFORESET = 0;
958         ScidRegs.SCIFFRX.bit.RXFIFORESET = 1;
959     } else {
960         RXDdata = RXDdata & 0x00FF;
961         if (com_state == 0) {
962             if (RXDdata == '*') {
963                 com_state = 1; //for receiving path
964             } else if (RXDdata == '0') {
965                 com_state = 2; //for receiving optitrack data
966             } else {
967                 com_state = 0;
968             }
969         } else if (com_state == 1) {
970             if (RXDdata == '*') {
971                 com_state = 10; // into the read mode
972                 received_path_count = 0;
973                 newAstarPath = 0;
974             } else if (RXDdata == '$') {
975                 com_state = 30;
976                 received_LV_count = 0;
977             } else if (RXDdata == '!') {
978                 com_state = 40;
979                 received_CMD_count = 0;
980             } else {
981                 com_state = 0;
982             }
983         } else if (com_state == 10) { // this should not occurred left it here for future code with A* path planning
984             path_received[received_path_count] = RXDdata;
985             received_path_count++;
986             if (received_path_count >= 81) {
987                 received_path_count = 0;
988                 com_state = 0;
989                 newAstarPath = 1;
990             }
991         } else if (com_state == 2) {
992             if (RXDdata == 'p'){
993                 com_state = 20;
994             } else {
995                 com_state = 0;
996             }
997         } else if (com_state == 20) {
998             if (RXDdata == 't'){

```

```

999         com_state = 21;
1000     } else {
1001         com_state = 0;
1002     }
1003 } else if (com_state == 21) {
1004     if (RXDdata == 'i'){
1005         com_state = 22;
1006     } else {
1007         com_state = 0;
1008     }
1009 } else if (com_state == 22) {
1010     if (RXDdata == '.'){
1011         com_state = 23;
1012         // received_opti_count = 0;
1013     } else {
1014         com_state = 0;
1015     }
1016 } else if (com_state == 23) {
1017     if (received_opti_count % 2 == 0){ // this is for motion tracking system left for future code
1018         tempLSB = RXDdata;
1019     } else {
1020         optirx.rawData[received_opti_count/2] = (RXDdata << 8)|tempLSB;
1021     }
1022     received_opti_count++;
1023     if (received_opti_count >= OPTITRACKDATASIZE*4) {
1024         if (new_optitrack == 0) {
1025             for (i=0;i<OPTITRACKDATASIZE;i++) { // x, y, heading, trackableID, framecount
1026                 Optitrackdata[i] = optirx.floatData[i];
1027             }
1028             new_optitrack = 1;
1029         }
1030         received_opti_count = 0;
1031         com_state = 0;
1032     }
1033 } else if (com_state == 30) {
1034     if ((received_LV_count % 2) == 0) {
1035         tempLSB = RXDdata;
1036     } else {
1037         DataFromLabView.rawData[received_LV_count/2] = (RXDdata << 8)|tempLSB;
1038     }
1039     received_LV_count++;
1040     if (received_LV_count >= 4*LVNUM_TOFROM_FLOATS) {
1041         received_LV_count = 0;
1042         com_state = 0;
1043         fromLVvalues[0] = DataFromLabView.floatData[0];
1044         fromLVvalues[1] = DataFromLabView.floatData[1];
1045         fromLVvalues[2] = DataFromLabView.floatData[2];
1046         fromLVvalues[3] = DataFromLabView.floatData[3];
1047         fromLVvalues[4] = DataFromLabView.floatData[4];
1048         fromLVvalues[5] = DataFromLabView.floatData[5];
1049         fromLVvalues[6] = DataFromLabView.floatData[6];
1050         fromLVvalues[7] = DataFromLabView.floatData[7];
1051         NewLVData = 1; // Flag new data
1052     }
1053 } else if (com_state == 40) {
1054     if ((received_CMD_count % 2) == 0) {
1055         tempLSB = RXDdata;
1056     } else {
1057         DataFromLinuxCMD.rawData[received_CMD_count/2] = (RXDdata << 8)|tempLSB;
1058     }
1059     received_CMD_count++;
1060     if (received_CMD_count >= 4*CMDNUM_FROM_FLOATS) {
1061         received_CMD_count = 0;
1062         com_state = 0;
1063
1064         LinuxCommands[0] = DataFromLinuxCMD.floatData[0];
1065         LinuxCommands[1] = DataFromLinuxCMD.floatData[1];
1066         LinuxCommands[2] = DataFromLinuxCMD.floatData[2];
1067         LinuxCommands[3] = DataFromLinuxCMD.floatData[3];
1068         LinuxCommands[4] = DataFromLinuxCMD.floatData[4];
1069         LinuxCommands[5] = DataFromLinuxCMD.floatData[5];
1070         LinuxCommands[6] = DataFromLinuxCMD.floatData[6];
1071         LinuxCommands[7] = DataFromLinuxCMD.floatData[7];
1072         LinuxCommands[8] = DataFromLinuxCMD.floatData[8];
1073         LinuxCommands[9] = DataFromLinuxCMD.floatData[9];
1074         LinuxCommands[10] = DataFromLinuxCMD.floatData[10];
1075         newLinuxCommands = 1; // Flag new data
1076     }
1077 }
1078 }
1079 }
1080 numRXD ++;
1081 ScidRegs.SCIFFRX.bit.RXFFINTCLR = 1;
1082 PieCtrlRegs.PIEACK.all = PIEACK_GROUP8;
1083 }
1084
1085
1086 // SerialA only setup for Tera Term connection
1087 char serial_printf_bufSCIA[BUF_SIZE_SCIA];
1088
1089 uint16_t serial_printf(serialSCIA_t *s, char *fmt, ...)

```

```

1090 {
1091     va_list ap;
1092
1093     va_start(ap,fmt);
1094     vsprintf(serial_printf_bufSCIA,fmt,ap);
1095     va_end(ap);
1096
1097     return serial_sendSCIA(s,serial_printf_bufSCIA,strlen(serial_printf_bufSCIA));
1098 }
1099
1100 //For Text LCD SCIB only
1101 char UART_printf_bufferSCIB[BUF_SIZEESCIB];
1102 void UART_vprintfLine(unsigned char line, char *format, va_list ap)
1103 {
1104     char sendmsg[24];
1105
1106     int i;
1107
1108     vsprintf(UART_printf_bufferSCIB,format,ap);
1109
1110     // Add header information and pad end of transfer with spaces to clear display
1111     sendmsg[0] = 0xFE;
1112     sendmsg[1] = 'G';
1113     sendmsg[2] = 1;
1114     sendmsg[3] = line;
1115     for (i=4;i<24;i++) {
1116         if (i >= strlen(UART_printf_bufferSCIB)+4) {
1117             sendmsg[i] = ' ';
1118         } else {
1119             sendmsg[i] = UART_printf_bufferSCIB[i-4];
1120         }
1121     }
1122     serial_sendSCIB(&SerialB,sendmsg,24);
1123 }
1124
1125 void UART_printfLine(unsigned char line, char *format, ...)
1126 {
1127     va_list ap;
1128     va_start(ap, format);
1129     UART_vprintfLine(line,format,ap);
1130 }

```